Kishore S

CH.SC.U4CSE24222

Week – 8

Design and Analysis of Algorithm(23CSE211)

Huffman Coding

Write a c program for Huffman Coding

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_TREE_HT 100
#define MAX_CHAR 256
struct MinHeapNode {
    char data;
    unsigned freq;
    struct MinHeapNode *left, *right;
};
struct MinHeap {
    unsigned size;
    unsigned capacity;
    struct MinHeapNode** array;
};
struct MinHeapNode* newNode(char data, unsigned freq) {
    struct MinHeapNode* temp = (struct MinHeapNode*)malloc(sizeof(struct
MinHeapNode));
    temp->left = temp->right = NULL;
    temp->data = data;
    temp->freq = freq;
    return temp;
}
struct MinHeap* createMinHeap(unsigned capacity) {
    struct MinHeap* minHeap = (struct MinHeap*)malloc(sizeof(struct
MinHeap));
    minHeap->size = 0;
    minHeap->capacity = capacity;
```

```
    minHeap->array = (struct MinHeapNode**)malloc(minHeap->capacity *
sizeof(struct MinHeapNode*));
    return minHeap;
}
void swapMinHeapNode(struct MinHeapNode** a, struct MinHeapNode** b) {
    struct MinHeapNode* t = *a;
    *a = *b;
    *b = t;
}
void minHeapify(struct MinHeap* minHeap, int idx) {
    int smallest = idx;
    int left = 2 * idx + 1;
    int right = 2 * idx + 2;

    if (left < minHeap->size && minHeap->array[left]->freq < minHeap-
>array[smallest]->freq)
        smallest = left;

    if (right < minHeap->size && minHeap->array[right]->freq < minHeap-
>array[smallest]->freq)
        smallest = right;

    if (smallest != idx) {
        swapMinHeapNode(&minHeap->array[smallest], &minHeap-
>array[idx]);
        minHeapify(minHeap, smallest);
    }
}
struct MinHeapNode* extractMin(struct MinHeap* minHeap) {
    struct MinHeapNode* temp = minHeap->array[0];
    minHeap->array[0] = minHeap->array[minHeap->size - 1];
```

```
    --minHeap->size;
    minHeapify(minHeap, 0);
    return temp;
}
void insertMinHeap(struct MinHeap* minHeap, struct MinHeapNode*
minHeapNode) {
    ++minHeap->size;
    int i = minHeap->size - 1;
    while (i && minHeapNode->freq < minHeap->array[(i - 1) / 2]->freq) {
        minHeap->array[i] = minHeap->array[(i - 1) / 2];
        i = (i - 1) / 2;
    }
    minHeap->array[i] = minHeapNode;
}
void buildMinHeap(struct MinHeap* minHeap) {
    int n = minHeap->size - 1;
    for (int i = (n - 1) / 2; i >= 0; --i)
        minHeapify(minHeap, i);
}
int isLeaf(struct MinHeapNode* root) {
    return !(root->left) && !(root->right);
}
void printCodes(struct MinHeapNode* root, int arr[], int top) {
    if (root->left) {
        arr[top] = 0;
        printCodes(root->left, arr, top + 1);
    }
    if (root->right) {
        arr[top] = 1;
        printCodes(root->right, arr, top + 1);
```

```
    }
    if (isLeaf(root)) {
        printf("  %c    |    %3d    | ", root->data, root->freq);
        for (int i = 0; i < top; ++i)
            printf("%d", arr[i]);
        printf("\n");
    }
}
void calculateSpaceSaving(struct MinHeapNode* root, int arr[], int top, int* totalOriginalBits, int* totalCompressedBits) {
    if (root->left) {
        arr[top] = 0;
        calculateSpaceSaving(root->left, arr, top + 1, totalOriginalBits, totalCompressedBits);
    }
    if (root->right) {
        arr[top] = 1;
        calculateSpaceSaving(root->right, arr, top + 1, totalOriginalBits, totalCompressedBits);
    }
    if (isLeaf(root)) {
        *totalOriginalBits += root->freq * 8; // Assuming 8 bits per character
        *totalCompressedBits += root->freq * top;
    }
}
void HuffmanCodes(char data[], int freq[], int size) {
    struct MinHeapNode *left, *right, *top;
    struct MinHeap* minHeap = createMinHeap(size);
    for (int i = 0; i < size; ++i)
        minHeap->array[i] = newNode(data[i], freq[i]);
    minHeap->size = size;
```

```
    buildMinHeap(minHeap);
    while (minHeap->size != 1) {
        left = extractMin(minHeap);
        right = extractMin(minHeap);
        top = newNode('$', left->freq + right->freq);
        top->left = left;
        top->right = right;
        insertMinHeap(minHeap, top);
    }
    int arr[MAX_TREE_HT], topIdx = 0;
    printf("\nChar | Freq | Huffman Code\n");
    printf("-------------------------\n");
    struct MinHeapNode* root = extractMin(minHeap);
    printCodes(root, arr, topIdx);
    int totalOriginalBits = 0, totalCompressedBits = 0;
    calculateSpaceSaving(root, arr, 0, &totalOriginalBits,
&totalCompressedBits);
    printf("\nSpace Saving Analysis:\n");
    printf("----------------------\n");
    printf("Original size (8 bits per character): %d bits\n", totalOriginalBits);
    printf("Compressed size (Huffman coding): %d bits\n",
totalCompressedBits);
    printf("Space saved: %d bits (%.2f%%)\n",
           totalOriginalBits - totalCompressedBits,
           ((float)(totalOriginalBits - totalCompressedBits) / totalOriginalBits) *
100);
}
int main() {
    printf("CH.SC.U4CCSE24222\n");
    char str[1000];
    int freq[MAX_CHAR] = {0};
```

```
    printf("Enter the string: ");
    scanf("%s", str);
    for (int i = 0; str[i] != '\0'; i++) {
        freq[(unsigned char)str[i]]++;
    }
    char unique_chars[MAX_CHAR];
    int unique_freqs[MAX_CHAR];
    int n = 0;
    for (int i = 0; i < MAX_CHAR; i++) {
        if (freq[i] > 0) {
            unique_chars[n] = (char)i;
            unique_freqs[n] = freq[i];
            n++;
        }
    }
    HuffmanCodes(unique_chars, unique_freqs, n);
    return 0;
}
```

OUTPUT:

```
kishore@kishore-LOQ-15IRX9: /media/kishore/Kishore Edu/Studies/sem 4/DAA/week 8

kishore@kishore-LOQ-15IRX9:/media/kishore/Kishore Edu/Studies/sem 4/DAA/week 8$ gcc Huffman_Coding.c -o one
kishore@kishore-LOQ-15IRX9:/media/kishore/Kishore Edu/Studies/sem 4/DAA/week 8$ ./one
CH.SC.U4CCSE24222
Enter the string: onepiece

Char | Freq | Huffman Code
--------------------------
  e  |    3   | 0
  c  |    1   | 100
  p  |    1   | 101
  i  |    1   | 110
  o  |    1   | 1110
  n  |    1   | 1111

Space Saving Analysis:
----------------------
Original size (8 bits per character): 64 bits
Compressed size (Huffman coding): 20 bits
Space saved: 44 bits (68.75%)
kishore@kishore-LOQ-15IRX9:/media/kishore/Kishore Edu/Studies/sem 4/DAA/week 8$
```